

Generative AI with Diffusion

An introduction to Diffusion Models

Politecnico di Milano | 13/05/2026

Vincenzo Pomponi, PhD Student

*Automation Robotics and Machines Laboratory (ARM)
Institute of System and Technologies for Sustainable Production (ISTePS)*

Lectures overview

- Lecture 1 (06/05/2026):
 - From generation to sampling
 - Diffusion Models introduction
 - Forward Diffusion Process (closed form solution derivation)
 - Reverse Diffusion Process (training loss derivation)
- Lecture 2 (today):
 - Recap of Lecture 1
 - DDPM vs DDIM
 - Diffusion Model architectures
 - Practical Python coding

Today overview

- Recap lecture 1:
 - Closed-form solution of the Forward Diffusion Process.
 - Intractability of the true reverse diffusion process.
 - Solution for the posterior distribution of the reverse process when conditioned on x_0 .
 - Definition of the ELBO.
 - Derivation of loss function.
- Sampling strategies:
 - DDPM
 - DDIM
- Diffusion Models architectures examples
- Practical Python coding

Section 1: **From Generation to Sampling**

Goal: Formalize what it means to “generate” something.

Input Data

Images:

- Height H and Width W
- 3 color channels (RGB)



$$\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$$

Videos:

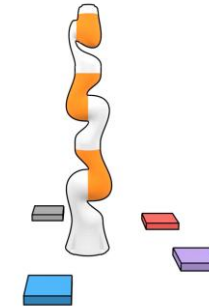
- T time frames
- Each frame is an image



$$\mathbf{x} \in \mathbb{R}^{T \times H \times W \times 3}$$

Robot trajectories:

- N robot configurations
- K DOFs + 1 gripper



$$\mathbf{x} \in \mathbb{R}^{N \times (K+1)}$$

We represent the objects we want to generate as vectors:

$$\mathbf{x} \in \mathbb{R}^d$$

Generation

Generation as sampling from the data distribution

Data distribution: distribution of objects we want to generate:

 p_{data}

Probability density:

$$p_{\text{data}} : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0},$$
$$\mathbf{x} \mapsto p_{\text{data}}(\mathbf{x})$$

Generation means sampling from the data distribution:

$$\mathbf{x} \sim p_{\text{data}}$$



$$\mathbf{x} =$$



Section 2: Diffusion Models

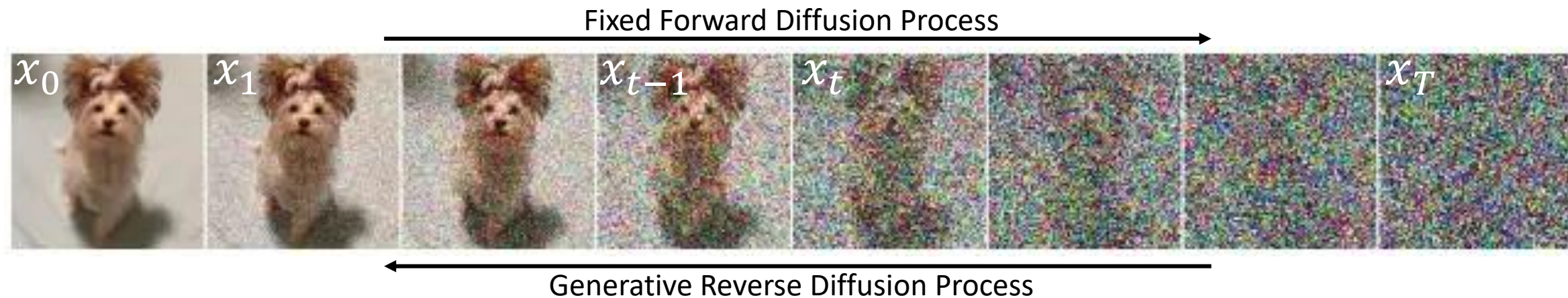
Goal: Define what are diffusion models

Diffusion Models

A Diffusion Model learns how to turn random noise into realistic data by gradually reversing a process that adds noise to real data.

Diffusion Models work in two main steps:

- 1. Forward Diffusion Process:** gradually add noise to real data (e.g. images) until it becomes completely random, like pure Gaussian noise.
- 2. Reverse Diffusion Process:** the model learns how to reverse this process.



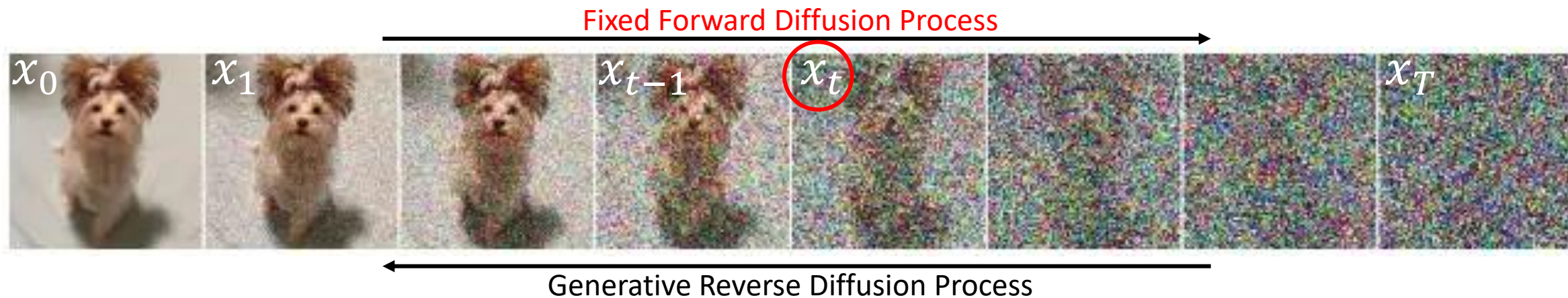
Section 2a: **Forward Diffusion Process**

Goal: mathematically define the forward diffusion process

Forward Diffusion Process

By recursively applying the previous procedure:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$
$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$



Forward Diffusion Process

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$

The previous expression shows that:

- $\sqrt{\bar{\alpha}_t}$ controls how much of the original signal $\mathbf{x}_0$ is preserved.
- $\sqrt{1 - \bar{\alpha}_t}$ determines the amount of accumulated noise.

Equivalently, we can write the closed-form equation as:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

This means that, given a noise schedule $\{\beta_t \in (0, 1)\}_{t=1}^T$, the noisy sample at any diffusion step can be computed in closed form using the equation above.

Section 2b:

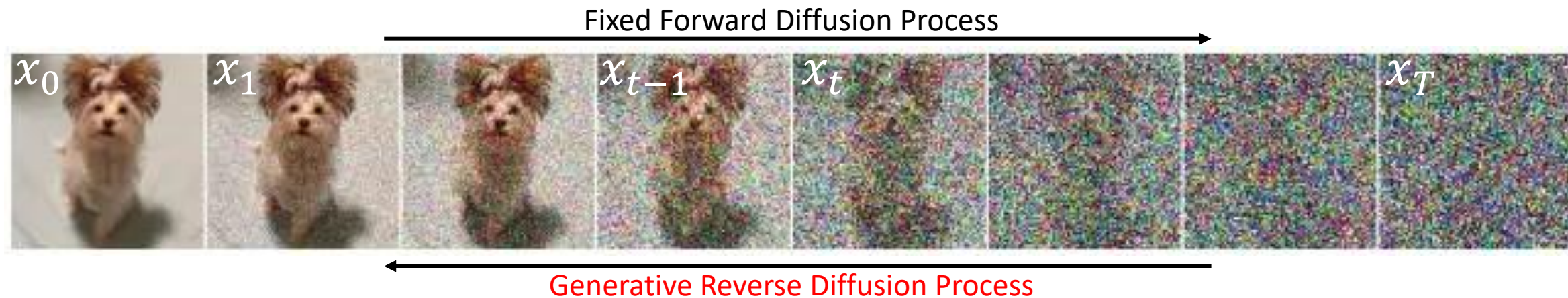
Reverse Diffusion Process

Goal: derive the loss function used for training diffusion models

Reverse Diffusion Process

The reverse diffusion process aims to invert the forward noising procedure by transforming a simple Gaussian noise distribution into the data distribution through a sequence of denoising steps.

Starting from a noise sample $x_T \sim \mathcal{N}(0, I)$, the goal is to progressively recover a structured data sample x_0 by iteratively reconstructing the vector x_{t-1} from x_t .



Reverse Diffusion Process

We can demonstrate that the reverse conditional distribution is intractable:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$

We have that both $q(\mathbf{x}_{t-1})$ and $q(\mathbf{x}_t)$ implicitly depend on $q(\mathbf{x}_0)$ which is unknown and intractable.

Consequently, $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is also intractable!

$q(\mathbf{x}_0)$ denotes the true (unknown) data distribution from which the training samples are generated and we only can observe samples from it.

Reverse Diffusion Process

We have demonstrated that the reverse conditional distribution is tractable when conditioned on $\mathbf{x}_0$ and can be approximated with a Gaussian distribution with mean $\tilde{\boldsymbol{\mu}}_t$ and covariance $\tilde{\boldsymbol{\beta}}_t \mathbf{I}$:

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\boldsymbol{\beta}}_t \mathbf{I})$$

Reverse Diffusion Process

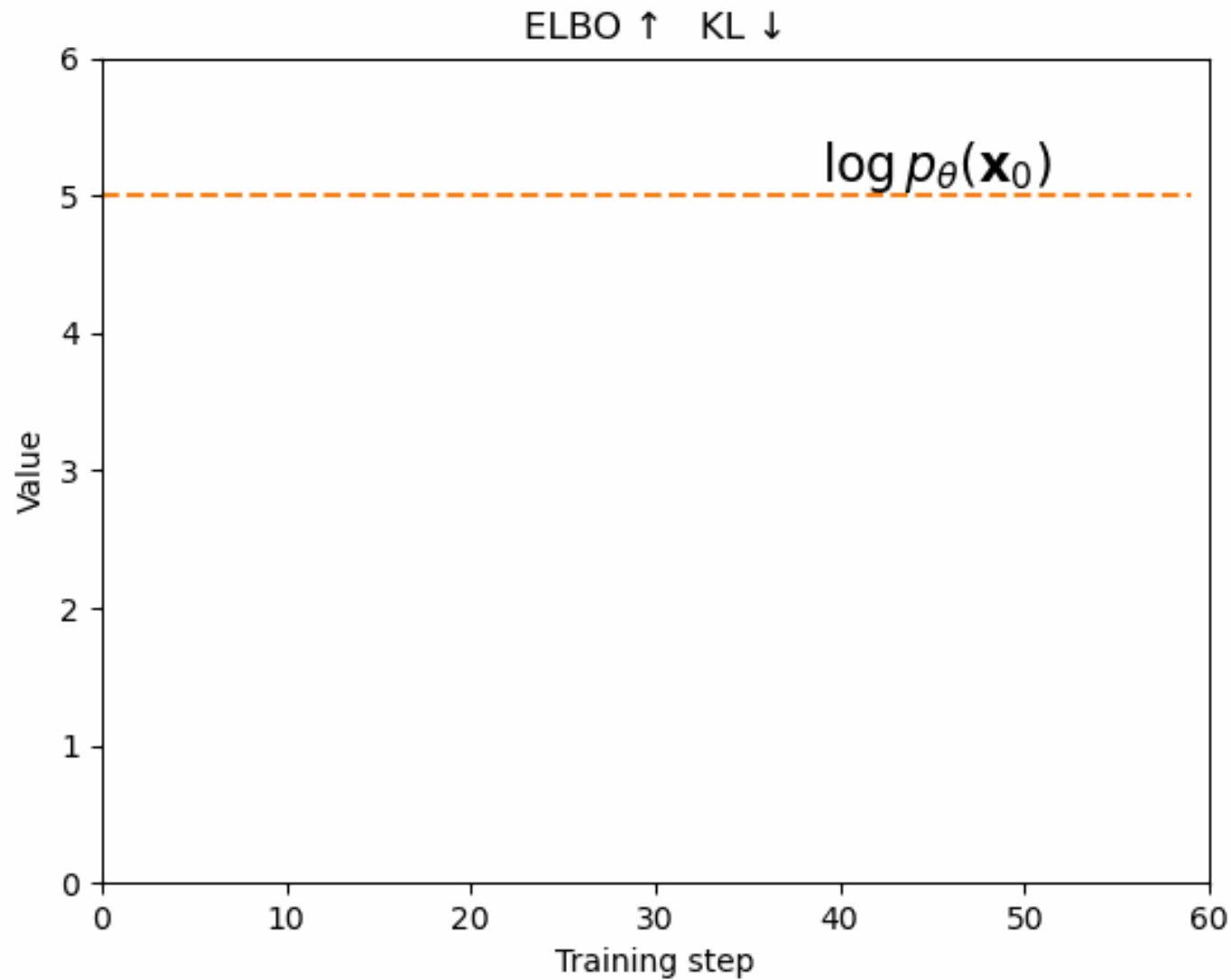


INTRACTABLE



$$p_{\theta}(x_0) = \int p_{\theta}(x_{0:T}) dx_{1:T}$$

Reverse Diffusion Process



Reverse Diffusion Process

$$L_{ELBO} = - \sum_{t=2}^T \mathbb{E} [D_{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t))] + \textit{CONSTANT}$$

Reverse Diffusion Process

The goal now is to maximize the ELBO, so that the $D_{KL}(q||p_\theta)$ will be minimized. In this way we are forcing the model posterior to match the forward process and to do so we train that model using the ELBO:

$$q(x_{t-1}|x_t, x_0) \approx p_\theta(x_{t-1}|x_t)$$

Reverse Diffusion Process

To recover the training loss, we focus on one single KL term:

$$D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))$$

Both distributions are Gaussian:

- True posterior: $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t \mathbf{I})$
- Model: $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I})$

For simplicity, the true posterior and the model have the same covariance $\tilde{\beta}_t$.

Substituting, we obtain:

$$D_{KL} = \frac{1}{2\tilde{\beta}_t} \|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t)\|^2$$

Reverse Diffusion Process

But, instead predicting the whole noisy image, it is demonstrated that predicting the noise is more efficient [9].

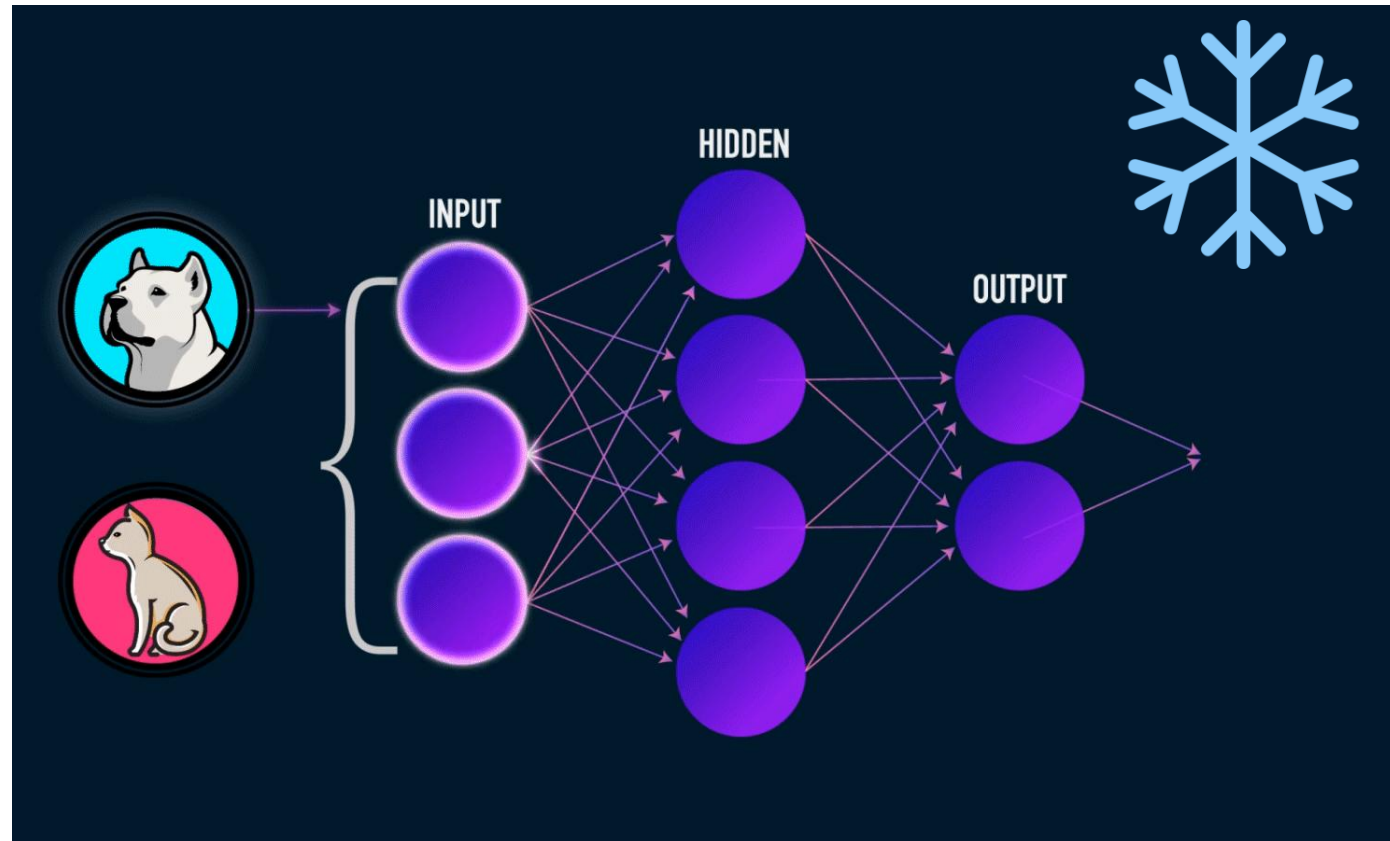
The loss function, then, becomes:

$$L = \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2$$

Section 3: Sampling strategies

Goal: How can we effectively generate samples from a trained diffusion model?

Trained Model



Sampling from a DM

Sampling from a diffusion model is an iterative process that transforms pure Gaussian noise into a coherent data sample by sequentially applying a learned reverse-denoising model to "undo" the noise added during training.

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$$

Sampling from a DM

The sampling process starts from:

$$\mathbf{x}_T \sim \mathcal{N}(0, I)$$

At each timestep t , the model receives the current noisy sample $\mathbf{x}_t$ and predicts the noise component:

$$\epsilon_{\theta}(\mathbf{x}_t, t)$$

The key idea is that, if the model can accurately estimate the noise contained in $\mathbf{x}_t$, then we can subtract that noise and recover a cleaner version of the sample.

Sampling Strategies

- DDPM (Denoising Diffusion Probabilistic Models)
 - Follow the original stochastic reverse process derived from the probabilistic formulation.
 - Alternate between a deterministic denoising operation and a stochastic sampling operation.
 - Higher number of diffusion steps.
 - Less efficient.
- DDIM (Denoising Diffusion Implicit Models)
 - The key idea is that the reverse diffusion process does not necessarily need to be fully stochastic.
 - Non-Markovian reverse process.
 - Preserve the same marginal distributions.
 - Less number of diffusion steps.
 - More efficient.

Section 3a:

DDPM: Denoising Diffusion Probabilistic Model

Goal: How can we effectively generate samples from a trained diffusion model using DDPM sampling strategy?

DDPM

Recall the forward diffusion process:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$

$\mathbf{x}_t$ is obtained by combining:

- A scaled version of the original signal $\mathbf{x}_0$.
- An additive Gaussian noise component $\boldsymbol{\epsilon}$.

DDPM

If the prediction is accurate, we can isolate the clean signal from the closed-form solution of the forward diffusion process:

$$\hat{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t) \right)$$

Predicting the noise is therefore equivalent to estimating the original clean sample.

DDPM

Instead of directly using the estimate $\hat{x}_0$ as the final generated sample, diffusion models use it to construct the reverse transition distribution:

$$p_{\theta}(x_{t-1}|x_t),$$

Which generates a slightly less noisy sample x_{t-1} from the current noisy state x_t .

Recall the analytical mean of the reverse posterior of $q(x_{t-1}|x_t, x_0)$:

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} x_0$$

DDPM

Combining the expressions of the estimate $\hat{x}_0$ and the analytical mean of the true posterior of the reverse diffusion process, yields the parameterized reverse mean:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right).$$

The quantity $\mu_{\theta}(x_t, t)$ should not be interpreted as the final reconstructed sample. Instead, it represents the center (mean) of the Gaussian distribution used to generate the next denoising step.

DDPM

The reverse transition is therefore modelled as:

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I})$$

Where:

- $\boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t)$ determines the denoising direction,
- $\tilde{\beta}_t \mathbf{I}$ controls the stochasticity of the reverse process.

Sampling from the Gaussian gives:

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$$

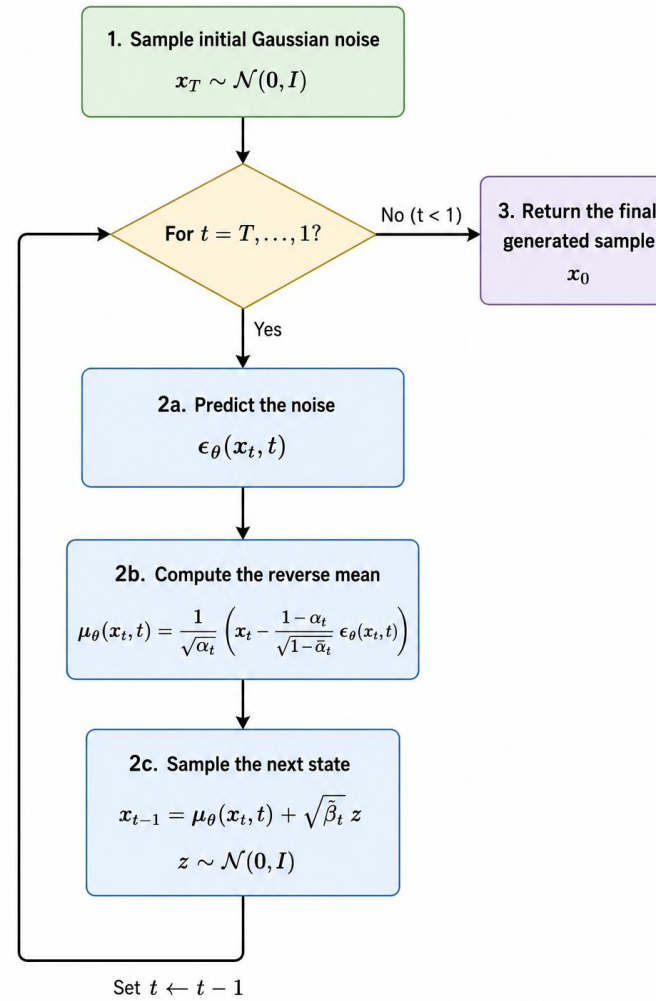
DDPM

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$$

This equation reveals the 2 fundamental components of DDPM sampling:

- **Deterministic denoising component:** The term $\boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t)$ is entirely determined by the neural network prediction $\boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)$. Therefore, the denoising direction is deterministic once the network output is fixed.
- **Stochastic sampling component:** The term $\sqrt{\tilde{\beta}_t} \mathbf{z}$, instead, introduces randomness into the reverse process. The variance parameter $\tilde{\beta}_t$ controls the amount of randomness injected at each reverse step.

DDPM



Section 3b:

DDIM: Denoising Diffusion Implicit Model

Goal: How can we effectively generate samples from a trained diffusion model using DDIM sampling strategy?

DDIM

Denoising Diffusion Implicit Models (DDIMs) were introduced to accelerate the sampling procedure of diffusion models while preserving the same training objective used in DDPM.

The key idea behind DDIM is that the reverse diffusion process does not necessarily need to be fully stochastic.

DDIM constructs a *non-Markovian* reverse process that preserves the statistical relationship between x_0 and x_t defined by the original forward process:

$$q(x_t|x_0)$$

DDIM

The starting point is the forward diffusion equation:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$

As in DDPM, the neural network predicts the noise:

$$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$$

Using this prediction, we can estimate the original clean sample $\mathbf{x}_0$ by rearranging the forward diffusion equation:

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right)$$

DDIM

Once $\hat{\mathbf{x}}_0$ is estimated, DDIM defines the reverse update as:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$$

This expression can be interpreted as a combination of 3 terms:

- $\sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0$: preserves the reconstructed clean signal.
- $\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)$: controls the direction of the denoising trajectory using the predicted noise.
- $\sigma_t \mathbf{z}$: optionally injects additional stochasticity

DDIM

Once $\hat{x}_0$ is estimated, DDIM defines the reverse update as:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_{\theta}(x_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I)$$

Unlike DDPM, the amount of randomness is now explicitly controlled by the parameter σ_t .

If $\sigma_t = 0$, no Gaussian noise is sampled during the reverse process, and consequently the entire generation trajectory is uniquely determined by:

- The noise sample x_T .
- The neural network predictions.

Thanks for the attention



Feel free to contact me if you have any questions or curiosity 😊
vincenzo.pomponi@supsi.ch